

Applied Machine Learning

Linear Regression Models



UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

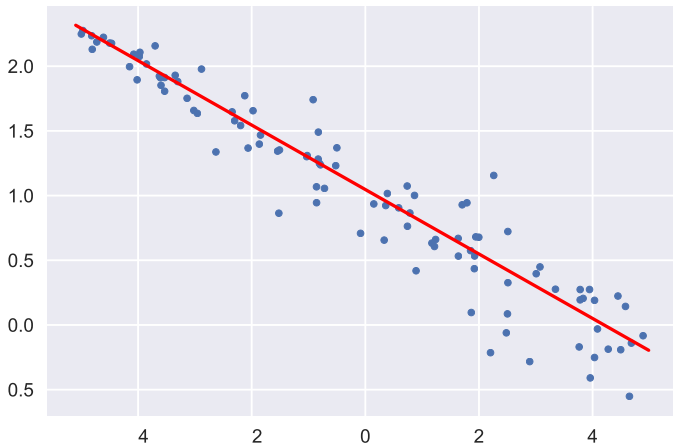
`richard.johansson@cse.gu.se`

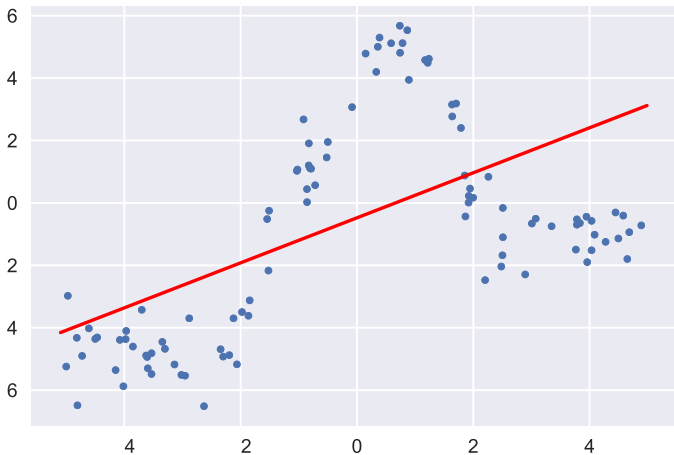
linear regression models

- ▶ a **linear regression** model predicts numerical output values like this:

$$y = \mathbf{w} \cdot \mathbf{x}$$

- ▶ explanation of the parts:
 - ▶ again, $\mathbf{x}$ is an encoded feature vector
 - ▶ $\mathbf{w}$ is a vector representing relationships between features and the output y





training linear regression models

- ▶ just like we discussed for linear classifiers, there are many training algorithms for linear regression
- ▶ from previous courses, you are probably familiar with **least-squares regression**

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w} \cdot \mathbf{x}_i)^2$$

recall: analytic solution to least squares regression

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

recall: analytic solution to least squares regression

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

- ▶ not the way it's implemented in scikit-learn: any idea why?

the Widrow-Hoff algorithm

- ▶ almost like the perceptron!
- ▶ NB the “learning rate” η

$\mathbf{w} = (0, \dots, 0)$

repeat N times

for $(\mathbf{x}_i, y_i)$ in the training set

$g = \mathbf{w} \cdot \mathbf{x}_i$

error = $g - y_i$

$\mathbf{w} = \mathbf{w} - \eta \cdot \text{error} \cdot \mathbf{x}_i$

return $\mathbf{w}$

linear regressors in scikit-learn

- ▶ small selection of learning algorithms for linear regressors:
 - ▶ `sklearn.linear_model.LinearRegression` (least squares)
 - ▶ `sklearn.linear_model.Ridge` (later)
 - ▶ `sklearn.linear_model.Lasso` (later)
 - ▶ `sklearn.svm.LinearSVR` (later)
- ▶ after training, weights are stored in the attribute `model.coef_`